

Discovery Executive Summary

Project: discovery-26-aug · Generated: 26/08/2026, 16:42:35

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Testing & Quality Assurance Analysis	—
2	Performance & Sustainability Analysis	—

1. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

PingCRM's test suite is critically under-resourced across both its Laravel backend and React/TypeScript frontend. Of 141 PHP source files and 903 frontend TypeScript/TSX files, only 5 test files exist in total — two real backend feature tests (Contacts, Organizations), one PHP unit placeholder that asserts `assertTrue(true)`, one frontend smoke test that asserts `expect(true).toBe(true)`, and a shared TestCase base class. Estimated overall coverage is well below 5%. Entire high-risk domains ship with zero test protection: the authentication flow, the Users module, Reports, and the entire IVR platform (83 controllers, 12 legacy GodServices, 12 repositories); no contract tests exist for any API endpoint or JSON response schema. Frontend Vitest tests are absent from the CI pipeline entirely, meaning the React layer has no automated regression gate. The two real backend feature tests run on CI but no evidence of being a required status check was found, compounding the risk.

§5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests (target 0)	0	1–3	>3	Auth, Users, Reports, IVR (83 controllers), 12 Legacy GodServices, 12 Repositories — >10 modules	High Risk
H2	Low Test Coverage	Overall coverage % (target >80%)	>80%	50–80%	<50%	Backend ~2% · Frontend ~0% (test-to-source ratio estimate — no coverage report present)	High Risk
H3	Missing Integration Tests	Key service/data boundaries covered % (target >70%)	>70%	30–70%	<30%	2 of ~30+ real service/controller boundaries = ~7%	High Risk
H4	Missing Contract Tests	Public APIs/contracts with contract tests % (target >80%)	>80%	40–80%	<40%	0 contract tests for any endpoint	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count (target 0)	0	1–5	>5	0	Good
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	PHP tests run on PR/push; frontend Vitest absent from CI entirely	High Risk
H7	Trivial / Assertion-Free Tests (additional)	Tests with no meaningful assertion (target 0). Good: 0 · Moderate: 1 · High Risk: >1	0	1	>1	2 vanity tests (<code>ExampleTest.php</code> , <code>smoke.test.ts</code>) with <code>assertTrue(true)</code> / <code>expect(true).toBe(true)</code>	High Risk

§5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Write PHPUnit feature tests for <code>AuthenticatedSessionController</code> (login, logout, rate-limiting), <code>UsersController</code> (CRUD, owner flag), and at least one Index/Store test per IVR module (12 modules × 2 = 24 tests)	High Risk	Critical
H2 — Low Test Coverage	Enable PHPUnit coverage reporting in CI; set 30% initial gate in <code>phpunit.xml</code> ; configure <code>@vitepress/coverage-v8</code> with a 20% branch threshold for the frontend	High Risk	Critical
H3 — Missing Integration Tests	Extend <code>RefreshDatabase</code> feature tests to <code>UsersController</code> mutations and all IVR Index HTTP endpoints; add <code>LoginRequest</code> rate-limiter boundary test	High Risk	High
H4 — Missing Contract Tests	Add <code>AssertableInertia</code> prop-contract tests for all IVR Index pages asserting <code>rows</code> , <code>filters</code> , and <code>legacyMeta</code> keys; add JSON response shape assertions for <code>wantsJson()</code> branches in all 12 IVR modules	High Risk	High
H6 — No CI Test Gate (Frontend)	Add <code>npm test</code> (Vitest) and <code>npx tsc --noEmit</code> steps to <code>.github/workflows/tests.yml</code> ; mark the workflow as a required GitHub branch protection status check on <code>master</code>	High Risk	High
H7 — Trivial / Assertion-Free Tests	Replace <code>tests/Unit/ExampleTest.php</code> with a real <code>User</code> model unit test; replace <code>resources/js/test/smoke.test.ts</code> with a <code>@testing-library/react</code> component render test	High Risk	Medium

§5.5 Expected Outcomes

- Authentication and user-management regressions are caught by automated tests before they reach production, eliminating the risk of silent privilege escalation or account lockout.
 - The entire IVR platform (83 controllers, 12 GodServices) has a regression safety net in place before any modernization or refactoring begins, allowing teams to refactor with confidence.
 - Frontend Vitest and TypeScript checks run on every CI build, ensuring broken React components or type errors are caught at pull-request time rather than at runtime.
 - API and Inertia prop-contract tests prevent breaking changes to JSON response shapes from silently corrupting IVR frontend pages after any backend change.
 - A clearly visible, enforced coverage gate (30% initial, rising to 75%) gives the team a measurable quality trend and blocks coverage regressions from merging to `master` .
- ```
 "stop_reason": "end_turn", "session_id": "5341f125-6bcb-491c-951f-dfa4b663448a", "total_cost_usd": 1.3179066999999998, "usage": {
 "input_tokens": 27, "cache_creation_input_tokens": 78709, "cache_read_input_tokens": 1615969, "output_tokens": 23360, "server_tool_use": {
 "web_search_requests": 0, "web_fetch_requests": 0, "service_tier": "standard", "cache_creation": {
 "ephemeral_1h_input_tokens": 78709, "ephemeral_5m_input_tokens": 0, "inference_geo": "not_available", "iterations": {
 [{"input_tokens": 1, "output_tokens": 1832, "cache_read_input_tokens": 99363, "cache_creation_input_tokens": 533, "cache_creation": {
 "ephemeral_5m_input_tokens": 0, "ephemeral_1h_input_tokens": 533, "type": "message"}]}, "speed": "standard", "modelUsage": {"claude-haiku-4-5-20251001": {
 "inputTokens": 10301, "outputTokens": 16, "cacheReadInputTokens": 0, "cacheCreationInputTokens": 0, "webSearchRequests": 0, "costUSD": 0.010381, "contextWindow": 200000, "maxOutputTokens": 32000, "sonnet": 4-6": {
 "inputTokens": 27, "outputTokens": 23360, "cacheReadInputTokens": 1615969, "cacheCreationInputTokens": 78709, "webSearchRequests": 0, "costUSD": 1.3075256999999998, "contextWindow": 200000, "[]", "terminal_reason": "completed", "fast_mode_state": "off", "uuid": "f45bcf14-e71e-4c25-8b71-12cf90f5c409"}
 }
 }
 }
 }
 }
 }
 }
```

2. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

The pingcrm codebase exhibits **High Risk** performance and sustainability characteristics dominated by one critical pattern: 12 Legacy “God Service” classes (covering every IVR module from AgentDesk to QueueManagement) each contain **45+ workflow methods that unconditionally call `sleep(1)`** — a direct 1-second synchronous blocking delay injected into every write path, guaranteeing multi-second response times on any operation that invokes these services. This alone would cap throughput and inflate server-time cost to unacceptable levels.

Compounding this, **81 raw SQL queries using `SELECT * FROM <table>`** without column projection expose the application to unbounded payload growth as tables widen, while **229 legacy React monolith components and 147 class-based React widgets** are bundled eagerly without any code-splitting or lazy loading, ballooning the initial JavaScript payload. A static mutable `$sharedRuntimeCache` array — present in all 12 God Services — grows unboundedly across requests in long-running PHP-FPM workers and silently leaks per-tenant write data between requests. The CI pipeline has partial caching (Composer layer cached, npm `ci` uncached, no incremental frontend builds), and the Heroku single-dyno deploy with `QUEUE_CONNECTION=sync` provides no autoscaling and processes background work synchronously in the request thread. Database performance (N+1, missing indexes) and frontend caching are deferred to the Backend Modernization and Frontend Modernization reports respectively.

§7.1 Benchmark Ratings Summary

| #  | Hotspot              | Primary KPI                                                      | <span class="rating rating-good">Good | <span class="rating rating-moderate">Moderate | <span class="rating rating-high-risk">High Risk | Measured                                            | Rating                                          |
|----|----------------------|------------------------------------------------------------------|---------------------------------------|-----------------------------------------------|-------------------------------------------------|-----------------------------------------------------|-------------------------------------------------|
| P1 | Algorithm Efficiency | High-complexity algorithm sites                                  | 0                                     | 1–5                                           | >5                                              | 0 — no nested O(n²)+ loops found                    | <span class="rating rating-good">Good           |
| P2 | Database Performance | Deferred → Backend Modernization (H14/H10)                       | —                                     | —                                             | —                                               | See Backend Modernization                           | — (deferred)                                    |
| P3 | API Performance      | Response-latency hotspots (blocking chains / oversized payloads) | 0                                     | 1–5                                           | >5                                              | 540 blocking sleep(1) calls + 81 SELECT * endpoints | <span class="rating rating-high-risk">High Risk |

|     |                                      |                                                                   |           |         |                   |                                                                                          |                                                 |
|-----|--------------------------------------|-------------------------------------------------------------------|-----------|---------|-------------------|------------------------------------------------------------------------------------------|-------------------------------------------------|
| P4  | Memory Efficiency                    | High-memory sites (unbounded caches, full-load, retention/leaks)  | 0         | 1–3     | >3                | 12 God Services with unbounded static array caches                                       | <span class="rating rating-high-risk">High Risk |
| P5  | CPU Efficiency                       | CPU-intensive operations on hot paths                             | 0         | 1–5     | >5                | 1 — image processing on every request with no dedicated cache/CDN path                   | <span class="rating rating-moderate">Moderate   |
| P6  | Concurrency                          | Parallelizable CPU-bound work + pool sizing                       | 0         | 1–5     | >5                | 0 — no CPU-bound sequential work identified                                              | <span class="rating rating-good">Good           |
| P7  | Caching                              | Deferred → Backend Modernization H14 / Frontend Modernization H11 | —         | —       | —                 | See those reports                                                                        | — (deferred)                                    |
| P8  | Resource Utilization                 | Over-provisioned / idle resource configs                          | 0         | 1–3     | >3                | 1 — single always-on Heroku dyno; QUEUE_CONNECTION=sync; no autoscaling                  | <span class="rating rating-moderate">Moderate   |
| P9  | Network Efficiency                   | Excessive-traffic sites (chatty/duplicate calls, no compression)  | 0         | 1–5     | >5                | 81 SELECT * endpoints + 229 legacy components serialising full rows to debug pre output  | <span class="rating rating-high-risk">High Risk |
| P10 | Build Efficiency                     | Build/test pipeline efficiency                                    | efficient | partial | slow / no caching | Composer cached; npm ci uncached; 904 TSX/TS files with no code-splitting                | <span class="rating rating-moderate">Moderate   |
| P11 | Logging Efficiency                   | Excessive-logging sites in hot paths                              | 0         | 1–10    | >10               | 0 explicit hot-loop logs; LOG_LEVEL=debug default in .env.example                        | <span class="rating rating-moderate">Moderate   |
| P12 | Sustainability                       | Resource-optimization posture                                     | optimized | partial | wasteful          | Single always-on dyno, QUEUE_CONNECTION=sync, no autoscaling, no carbon-aware scheduling | <span class="rating rating-high-risk">High Risk |
| P13 | Uncancelled async fetch (additional) | Async fetch calls without AbortController / cleanup               | 0         | 1–5     | >5                | 147 class-based React widgets in componentDidMount with bare fetch() and no cancellation | <span class="rating rating-high-risk">High Risk |

## §7.5 Actions Required

| Hotspot                       | Action                                                                                                                                                                                                           | Rating                                          | Priority                                |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------|
| P3 — API Performance          | Remove all 540 <code>sleep(1)</code> calls from 12 legacy God Services; replace <code>SELECT *</code> with column-projected Eloquent queries; dispatch remote-sync logic to Laravel queue                        | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| P4 — Memory Efficiency        | Remove <code>public static \$sharedRuntimeCache</code> from all 12 God Services; replace with <code>Cache::put()</code> with TTL if cross-request state is genuinely needed                                      | <span class="rating rating-high-risk">High Risk | <span class="sev sev-critical">Critical |
| P13 — Uncancelled async fetch | Add <code>AbortController</code> / <code>componentWillUnmount</code> cleanup to all 147 class-based legacy React widgets; deduplicate sibling fetches using a shared parent or Inertia props                     | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| P9 — Network Efficiency       | Replace 81 <code>DB::select(\“select * from …\“)</code> with projected queries; remove <code>JSON.stringify({rows, legacyMeta})</code> debug pre blocks from 229 legacy monolith components; add gzip to Apache2 | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| P12 — Sustainability          | Switch <code>QUEUE_CONNECTION=redis</code> ; add <code>worker</code> dyno to Procfile; enable gzip in <code>public/.htaccess</code> ; configure <code>LOG_CHANNEL=stderr</code> for Heroku; evaluate autoscaling | <span class="rating rating-high-risk">High Risk | <span class="sev sev-high">High         |
| P5 — CPU Efficiency           | Add <code>Cache-Control: public, max-age=31536000</code> headers to Glide image responses; place CDN in front of <code>/images/</code> route                                                                     | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium     |
| P8 — Resource Utilization     | Configure Heroku autoscaling or add Heroku Scheduler to scale down during off-peak windows; evaluate separate worker dyno configuration                                                                          | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium     |
| P10 — Build Efficiency        | Add npm dependency cache step to CI workflow keyed on <code>package-lock.json</code> ; introduce <code>React.lazy()</code> route-based code-splitting for legacy IVR modules                                     | <span class="rating rating-moderate">Moderate   | <span class="sev sev-medium">Medium     |
| P11 — Logging Efficiency      | Change <code>LOG_LEVEL=warning</code> in <code>.env.example</code> ; switch to <code>LOG_CHANNEL=stderr</code> for Heroku to avoid synchronous file I/O                                                          | <span class="rating rating-moderate">Moderate   | <span class="sev sev-low">Low           |

## §7.6 Expected Outcomes

- **Eliminating the 540 `sleep(1)` calls** will reduce IVR write response times from 1+ second to sub-50ms, unblocking Heroku PHP-FPM workers and increasing write path throughput by approximately 20x.
- **Removing unbounded static caches and projecting SQL columns** will stabilise PHP-FPM worker memory, eliminating crash-restart cycles under load and reducing per-request DB bandwidth by an estimated 60–80% on wide IVR tables.
- **Removing `JSON.stringify(rows)` debug blocks from 229 components and adding gzip compression** will reduce Inertia.js page payload size significantly, improving Time-to-Interactive and reducing bandwidth cost per page view.
- **Introducing a Redis queue with a worker dyno** decouples write-path latency from background processing, enabling HTTP responses to return immediately and allowing independent scaling of web and worker capacity based on actual load.

- Adding npm dependency caching and route-based code-splitting** will reduce CI build time and shrink the initial JS bundle, lowering client-side CPU/energy consumption per page visit and improving Core Web Vitals scores for all routes.
 

```

{
 "stop_reason": "end_turn",
 "session_id": "5d80ddcc-4ddf-4449-83d8-eed67682aa3b",
 "total_cost_usd": 2.1883325000000005,
 "usage": {
 "input_tokens": 40,
 "cache_creation_input_tokens": 118991,
 "cache_read_input_tokens": 3340385,
 "output_tokens": 30693,
 "server_tool_use": {
 "web_search_requests": 0,
 "web_fetch_requests": 0,
 "service_tier": "standard",
 "cache_creation": {
 "ephemeral_1h_input_tokens": 118991,
 "ephemeral_5m_input_tokens": 0,
 "inference_geo": "not_available",
 "iterations": [
 {
 "input_tokens": 1,
 "output_tokens": 2550,
 "cache_read_input_tokens": 139560,
 "cache_creation_input_tokens": 618,
 "cache_creation": {
 "ephemeral_5m_input_tokens": 0,
 "ephemeral_1h_input_tokens": 618,
 "type": "message"
 }
 },
 {
 "speed": "standard",
 "modelUsage": {
 "claude-haiku-4-5-20251001": {
 "inputTokens": 11676,
 "outputTokens": 16,
 "cacheReadInputTokens": 0,
 "cacheCreationInputTokens": 0,
 "webSearchRequests": 0,
 "costUSD": 0.011756,
 "contextWindow": 200000,
 "maxOutputTokens": 32000,
 "sonnet-4-6": {
 "inputTokens": 40,
 "outputTokens": 30693,
 "cacheReadInputTokens": 3340385,
 "cacheCreationInputTokens": 118991,
 "webSearchRequests": 0,
 "costUSD": 2.1765765000000004,
 "contextWindow": 200000,
 "terminal_reason": "completed",
 "fast_mode_state": "off",
 "uuid": "a72d4234-a865-41dd-893b-5a8a9ebd086c"
 }
 }
 }
 }
]
 }
 }
 }
}

```